

**TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ**



BÀI TẬP LỚN

ĐẢM BẢO CHẤT LƯỢNG VÀ KIỂM THỬ PHẦN MỀM

**ĐỀ TÀI: ĐÁNH GIÁ TESTABILITY CỦA MÃ NGUỒN VÀ
KIẾN TRÚC**

GIÁO VIÊN HƯỚNG DẪN : TS. NGUYỄN TUẤN LINH

HỌ VÀ TÊN : Vũ Việt Anh

LỚP : K58KTP.K01

MSSV : K225480106082

TRƯỜNG ĐHKTCN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

KHOA ĐIỆN TỬ

Độc lập - Tự do - Hạnh phúc

BÀI TẬP LỚN

Sinh viên: Vũ Việt Anh

MSSV: K225480106082

Lớp: K58KTP.K01

Ngành: Kỹ Thuật Phần Mềm

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Ngày giao đề: 01/06/2026

Ngày hoàn thành: 21/06/2026

Tên đề tài: Đánh giá Testability của mã nguồn và kiến trúc

Yêu cầu:

- Phân tích 4 đặc tính ảnh hưởng đến kiểm thử: *Observability*, *Controllability*, *Coupling* và *Cohesion*. Nghiên cứu bộ chỉ số hướng đối tượng (*CBO*, *WMC*, *LCOM*) và giải pháp từ *Kiến trúc Lục giác*.
- Cài đặt bài toán "Hệ thống thanh toán đơn hàng" (Python) qua 2 phiên bản đối chứng (Kiến trúc cũ và Kiến trúc Lục giác). Xây dựng bộ mã *Unit Test* sử dụng kỹ thuật *Mocks/Stubs* để giả lập lỗi DB và thời gian hệ thống trong môi trường cô lập.

GIÁO VIÊN HƯỚNG DẪN

(Kí và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN CHẤM

.....

.....

.....

.....

.....

.....

GIÁO VIÊN HƯỚNG DẪN

MỤC LỤC

LỜI MỞ ĐẦU	1
PHẦN 1. MỞ ĐẦU	2
1.1. Lý do chọn đề tài.....	2
1.2. Mục tiêu nghiên cứu.....	3
1.3. Phạm vi nghiên cứu.....	4
PHẦN 2. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN	5
2.1. Định nghĩa và các khái niệm cốt lõi.....	5
2.1.1. Định nghĩa về Khả năng kiểm thử	5
2.1.2. Khái niệm về Kiểm thử đơn vị.....	5
2.1.3. Khái niệm Mỗi nối kiểm thử và Sự cô lập	6
2.2. Phân tích chuyên sâu	6
2.2.1. Cơ chế tác động của cấu trúc kiến trúc đến bốn đặc tính Testability.....	6
2.2.2. Mối quan hệ giữa bộ chỉ số C&K và cấu trúc Unit Test.....	7
2.2.3. Giải pháp phá vỡ sự phụ thuộc bằng ranh giới Kiến trúc	
Lục giác	8
2.3. Sự liên hệ với chất lượng phần mềm	8
2.3.1. Vị trí của Testability trong mô hình chất lượng tiêu chuẩn	8
2.3.2. Giảm thiểu tỷ lệ sót lỗi và hiện thực hoá Shift-Left Testing.....	9
2.3.3. Kiểm soát Nợ kỹ thuật và thúc đẩy tính linh hoạt	9
PHẦN 3. VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ.....	11
3.1. Mô tả bài toán và ngữ cảnh thực tế	11
3.2. Áp dụng giải pháp	11
3.2.1. Mã nguồn phiên bản kiến trúc cũ phụ thuộc cứng	12
3.2.2. Mã nguồn phiên bản kiến trúc mới tối ưu hoá Testability	13
3.2.3. Bộ mã nguồn kiểm thử đơn vị cô lập sử dụng Mocks	17
3.3. Kết quả và Đánh giá.....	18
3.3.1. Mã nguồn chương trình đo lường tự động chỉ số cấu trúc.....	18

3.3.2. Kết quả thực thi chương trình	21
3.3.3. Phân tích kết quả và Đánh giá hiệu quả	22
PHẦN 4. KẾT LUẬN VÀ KIẾN NGHỊ.....	24
4.1. Tổng kết nội dung	24
4.2. Hạn chế của giải pháp	24
4.3. Định hướng mở rộng	25
TÀI LIỆU THAM KHẢO.....	27
MÃ QR VÀ LINK GITHUB BÀI LÀM	28

LỜI MỞ ĐẦU

Trong kỷ nguyên chuyển đổi số mạnh mẽ, các hệ thống phần mềm đóng vai trò cốt lõi trong mọi lĩnh vực của đời sống. Độ phức tạp ngày càng cao của các ứng dụng đặt ra yêu cầu sống còn đối với các nhà phát triển: Đảm bảo chất lượng phần mềm (SQA). Một lỗi kỹ thuật nhỏ khi vận hành không chỉ gây thiệt hại lớn về kinh tế mà còn ảnh hưởng nghiêm trọng đến uy tín chiến lược của doanh nghiệp.

Để tối ưu hóa quy trình, xu hướng kiểm thử sớm (Shift - Left Testing) ngày càng được chú trọng, trong đó Unit Testing đóng vai trò là chốt chặn đầu tiên. Tuy nhiên, hiệu quả của công tác này phụ thuộc rất lớn vào Khả năng kiểm thử (Testability) nội tại của kiến trúc và mã nguồn. Một thiết kế hướng tới Testability giúp lập trình viên dễ dàng cô lập thành phần, tự động hóa kịch bản kiểm thử nhanh chóng và tiết kiệm tối đa chi phí bảo trì hệ thống.

Nhận thức sâu sắc được mối liên hệ đó, em đã lựa chọn đề tài "Đánh giá testability của mã nguồn và kiến trúc" cho bài tập lớn này. Nghiên cứu tập trung bóc tách 4 thuộc tính cốt lõi bao gồm: *Observability*, *Controllability*, *Coupling* và *Cohesion*. Thông qua bài toán "Hệ thống thanh toán đơn hàng" bằng ngôn ngữ Python, đề tài xây dựng hai phiên bản đối chứng nhằm chứng minh hiệu quả của mô hình Kiến trúc Lục giác (Hexagonal Architecture), kỹ thuật giả lập (*Mocks/Stubs*) và bộ chỉ số hướng đối tượng (*CBO*, *WMC*, *LCOM*) trong việc giải phóng mã nguồn khỏi sự phụ thuộc cứng nhắc vào hạ tầng.

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy TS. Nguyễn Tuấn Linh. Những bài giảng tâm huyết, góc nhìn thực tiễn cùng sự định hướng tận tình của Thầy trong bộ môn Đảm bảo chất lượng và kiểm thử phần mềm đã giúp em tiếp cận tri thức một cách bài bản và khoa học. Bài tập lớn này chính là thành quả của quá trình học tập và nỗ lực dưới sự dìu dắt của Thầy. Do giới hạn về mặt kiến thức, bài làm khó tránh khỏi những thiếu sót, em rất mong nhận được những nhận xét, góp ý từ Thầy để đề tài được hoàn thiện hơn.

PHẦN 1. MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong quản trị dự án phần mềm hiện đại, việc phân phối sản phẩm nhanh chóng nhưng vẫn đảm bảo tính ổn định là một thách thức lớn. Theo học thuyết về kim tự tháp tự động hóa kiểm thử (Test Automation Pyramid) được phân tích bởi Matthew Heusser và Michael Larsen (2023), các bộ kiểm thử hộp đen từ giao diện (End-to-End/GUI Tests) thường rất chậm chạp, tốn kém chi phí vận hành và dễ gãy vỡ (brittle) trước các thay đổi nhỏ của hệ thống. Để giải quyết triệt để vấn đề này, xu hướng dịch chuyển sang kiểm thử hướng về phía lập trình viên (*Programmer-Facing Testing*) - cụ thể là Unit Testing - đóng vai trò vô cùng cốt lõi. Các bộ mã kiểm thử ở mức độ đơn vị nhỏ nhất này mang lại lợi thế vượt trội nhờ tốc độ thực thi nhanh, quy trình thiết lập kịch bản tinh gọn, ít biến động và dễ dàng tái hiện lỗi khi có sự cố xảy ra.

Tuy nhiên, rào cản lớn nhất khiến đội ngũ phát triển không thể triển khai Unit Testing hiệu quả chính là sự yếu kém trong Khả năng kiểm thử (Testability) nội tại của cấu trúc mã nguồn. Trong thực tế thiết kế hướng đối tượng thông thường (Naive OOP), các thành phần logic nghiệp vụ thường bị gắn chặt một cách cứng nhắc với hệ quản trị cơ sở dữ liệu hoặc API của bên thứ ba. Heusser và Larsen đã chỉ ra một nghịch lý kinh điển trong kỹ nghệ phần mềm: *"Lập trình viên chỉ muốn kiểm thử quả chuối, nhưng rốt cuộc họ buộc phải khởi tạo cả một khu rừng rậm xung quanh nó"* (*The Banana-Jungle Problem*). Hệ quả là một bài kiểm thử đơn vị đơn giản lại đòi hỏi phải kết nối cơ sở dữ liệu thật, tiêu tốn tài nguyên đám mây đắt đỏ và kéo dài thời gian phản hồi từ vài mili-giây lên thành nhiều giờ đồng hồ, làm mất đi ý nghĩa nguyên bản của Unit Testing.

Để tháo gỡ nút thắt về Testability, kiến trúc hệ thống cần phải thiết lập được các ranh giới rõ ràng (*clean lines*) giữa lõi nghiệp vụ và hạ tầng công nghệ. Mô hình Kiến trúc Lục giác (Hexagonal Architecture) xuất hiện như một giải pháp cứu cánh, cho phép cô lập hoàn toàn logic cốt lõi bên trong hình lục giác độc lập với thế giới bên ngoài. Khi cấu trúc đạt được sự phân tách tường minh, lập trình viên có thể dễ dàng sử dụng kỹ thuật thế thân kiểm thử (Test Doubles - bao gồm Mocks và Stubs) để tạo ra các điểm khớp nối linh hoạt (*seams*). Nhờ đó, chúng ta có thể kiểm thử toàn diện

phần lỗi của ứng dụng một cách độc lập mà không cần quan tâm đến sự hiện diện của hạ tầng thực tế.

Tại Việt Nam, việc áp dụng các mô hình kiến trúc tiên tiến để nâng cao Testability vẫn thường dừng lại ở mức độ lý thuyết định tính hoặc dựa trên cảm tính của lập trình viên. Nhằm mang lại một góc nhìn trực quan và có căn cứ khoa học, đề tài "Đánh giá testability của mã nguồn và kiến trúc" được lựa chọn để triển khai thực nghiệm trên ngôn ngữ Python. Thông qua bài toán "Hệ thống xử lý thanh toán đơn hàng", nghiên cứu sẽ bóc tách cấu trúc hệ thống dưới góc độ 4 thuộc tính (*Observability, Controllability, Coupling, Cohesion*), đồng thời áp dụng bộ chỉ số đo lường hướng đối tượng (*CBO, WMC, LCOM*) để chứng minh một cách định lượng bằng số liệu cụ thể về sức mạnh của Kiến trúc Lục giác và kỹ thuật Test Doubles trong việc tối ưu hóa năng lực kiểm thử phần mềm.

1.2. Mục tiêu nghiên cứu

Để đạt được mục tiêu tổng quát trên, đề tài tập trung hoàn thành bốn mục tiêu cụ thể sau:

- Hệ thống hóa lý thuyết về Testability: Làm rõ bản chất của bốn thuộc tính cấu thành năng lực kiểm thử (*Observability, Controllability, Coupling, Cohesion*) và cơ chế định lượng thiết kế thông qua bộ chỉ số hướng đối tượng (*CBO, WMC, LCOM*).
- Xây dựng mô hình thực nghiệm đối chứng: Thiết lập bài toán "Hệ thống xử lý thanh toán đơn hàng" trên ngôn ngữ Python với hai phiên bản kiến trúc: Kiến trúc cũ phụ thuộc cứng (Testability thấp) và Kiến trúc Lục giác cải tiến sử dụng Dependency Injection (Testability cao).
- Ứng dụng giải pháp thể thân kiểm thử: Triển khai bộ mã Unit Test tự động, áp dụng linh hoạt các kỹ thuật giả lập (*Mocks và Stubs*) để cô lập hoàn toàn logic nghiệp vụ cốt lõi, dễ dàng ép điều kiện biên và quan sát kết quả đầu ra.
- Đánh giá và chứng minh định lượng: Thực hiện đo lường các chỉ số cấu trúc giữa hai phiên bản, lập bảng so sánh trực quan nhằm đưa ra kết luận khoa học về tầm ảnh hưởng quyết định của thiết kế kiến trúc đối với tính dễ kiểm thử của phần mềm.

1.3. Phạm vi nghiên cứu

Để đảm bảo tính tập trung và chiều sâu cho nghiên cứu, phạm vi của đề tài được giới hạn cụ thể trong các khía cạnh sau:

- Về đối tượng nghiên cứu: Đề tài tập trung bóc tách thuộc tính Khả năng kiểm thử (Testability) của cấu trúc phần mềm dựa trên 4 đặc tính cốt lõi (*Observability, Controllability, Coupling, Cohesion*) và bộ 3 chỉ số hướng đối tượng thuộc tập chỉ số C&K bao gồm: *CBO* (Độ khớp nối đối tượng), *WMC* (Trọng số phương thức trên lớp), và *LCOM* (Sự thiếu gắn kết trong phương thức).
- Về nội dung thực nghiệm: Nghiên cứu được triển khai áp dụng cụ thể trên bài toán "Hệ thống xử lý thanh toán đơn hàng" (Order Payment Processor) với hai phiên bản kiến trúc đối chứng (Kiến trúc cũ phụ thuộc cứng và Kiến trúc Lục giác cải tiến).
- Về môi trường công nghệ và công cụ: Toàn bộ mã nguồn nghiệp vụ và mã kiểm thử được hiện thực hóa bằng ngôn ngữ lập trình Python, kết hợp với framework kiểm thử tiêu chuẩn unittest. Đề tài ứng dụng kỹ thuật thể thân kiểm thử (*Test Doubles* - cụ thể là *Mocks* và *Stubs*) để giả lập các thành phần hạ tầng (MySQL Database và API Email).
- Giới hạn của đề tài: Nghiên cứu chỉ tập trung giải quyết bài toán ở cấp độ Kiểm thử đơn vị (Unit Testing) trong môi trường mã nguồn cô lập. Các cấp độ kiểm thử khác như kiểm thử tích hợp (Integration Testing) trên môi trường cơ sở dữ liệu thật, kiểm thử hệ thống (System Testing) hay kiểm thử giao diện (UI Testing) không thuộc phạm vi nội dung của đề tài này.

PHẦN 2. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

2.1. Định nghĩa và các khái niệm cốt lõi

2.1.1. Định nghĩa về Khả năng kiểm thử

Theo tiêu chuẩn quốc tế về chất lượng phần mềm ISO/IEC 25010, khả năng kiểm thử là một đặc tính hiệu năng thuộc nhóm thuộc tính duy trì (Maintainability), thể hiện mức độ hiệu quả và dễ dàng mà một hệ thống hoặc module có thể được kiểm thử nhằm xác minh tính đúng đắn của các yêu cầu chức năng.

Dưới góc nhìn của kỹ nghệ phần mềm hiện đại, Matthew Heusser và Michael Larsen (2023) định nghĩa Testability là một thuộc tính thiết kế nội tại của mã nguồn. Một hệ thống có tính Testability cao khi và chỉ khi chi phí (về mặt thời gian, tài nguyên, công sức) bỏ ra để thiết lập kịch bản và thực thi các bộ kiểm thử là tối thiểu, nhưng lại mang lại độ tin cậy tối đa trong việc phát hiện khuyết tật ngầm. Testability không phải là một yếu tố có thể thêm vào sau khi mã nguồn đã hoàn thiện, mà nó là kết quả của một tư duy kiến trúc mạch lạc ngay từ giai đoạn định hình cấu trúc.

2.1.2. Khái niệm về Kiểm thử đơn vị

Kiểm thử đơn vị là cấp độ kiểm thử nhỏ nhất trong chiến lược đảm bảo chất lượng, tập trung vào việc xác minh tính đúng đắn của các thành phần mã nguồn độc lập (thường là một hàm, một phương thức hoặc một lớp) trong điều kiện cô lập hoàn toàn với các phần còn lại của hệ thống.

Một bài kiểm thử đơn vị tiêu chuẩn phải đáp ứng nguyên lý F.I.R.S.T:

- Fast (Nhanh): Thời gian chạy tính bằng phần nghìn giây.
- Independent (Độc lập): Các bài test không phụ thuộc kết quả hay thứ tự chạy của nhau.
- Repeatable (Tái lập): Chạy lại nhiều lần trên các môi trường khác nhau vẫn cho ra một kết quả duy nhất.
- Self-Validating (Tự xác thực): Tự đưa ra kết quả Đúng/Sai rõ ràng mà không cần con người đọc log.
- Timely (Kịp thời): Được viết song song hoặc ngay sau khi viết mã nghiệp vụ.

Năng lực Testability của kiến trúc đóng vai trò quyết định cấu trúc của Unit Test có đạt được tiêu chuẩn F.I.R.S.T này hay không.

2.1.3. Khái niệm Mỗi nối kiểm thử và Sự cô lập

Để thực hiện Unit Testing thành công, mã nguồn bắt buộc phải đạt được trạng thái Cô lập kiểm thử (Test Isolation). Điều này đồng nghĩa với việc loại bỏ hoàn toàn các tác động từ môi trường vật lý bên ngoài (như mạng Internet, hệ quản trị cơ sở dữ liệu, tệp tin hệ thống) ra khỏi phạm vi thực thi của hàm nghiệp vụ.

Để tạo ra sự cô lập này mà không cần sửa đổi cấu trúc logic cốt lõi, kiến trúc phần mềm cần ứng dụng khái niệm Mỗi nối (Seams). Mỗi nối là một vị trí cụ thể trong mã nguồn nơi mà một hành vi có thể thay đổi được mà không cần phải chỉnh sửa trực tiếp trên dòng mã đó. Trong lập trình hướng đối tượng, các ranh giới giao tiếp giữa các lớp nghiệp vụ và các lớp hạ tầng (ví dụ: Interface, Abstract Class) chính là những mối nối lý tưởng. Thông qua các mối nối này, lập trình viên có thể dễ dàng chèn vào các thành phần thể thân nhằm kiểm soát dữ liệu đầu vào và quan sát hành vi đầu ra của thành phần đang được kiểm thử.

2.2. Phân tích chuyên sâu

2.2.1. Cơ chế tác động của cấu trúc kiến trúc đến bốn đặc tính Testability

Để hiểu rõ tại sao một kiến trúc phần mềm lại bị coi là "khó kiểm thử", cần phân tích sâu cách thức mà các khuyết tật thiết kế phá hủy bốn thuộc tính nền tảng của Testability:

- Phá hủy tính kiểm soát (Controllability) do ràng buộc ngoại cảnh: Tính kiểm soát bị triệt tiêu hoàn toàn khi mã nguồn nghiệp vụ chứa các lời gọi trực tiếp đến các yếu tố động của môi trường, chẳng hạn như hàm lấy thời gian thực của hệ thống (`datetime.now()`) hoặc các truy vấn mạng trực tiếp. Khi logic giảm giá phụ thuộc vào ngày Thứ Sáu, lập trình viên không thể ép hệ thống chạy ca kiểm thử này vào ngày Thứ Ba nếu không can thiệp vào đồng hồ của hệ điều hành. Sự mất kiểm soát đầu vào này vi phạm nghiêm trọng nguyên lý *Repeatable* (Tính tái lập) của Unit Test.
- Suy giảm tính quan sát (Observability) bởi hiệu ứng phụ ngầm (Side-effects): Trong các thiết kế kém, nhiều phương thức thực hiện các nghiệp vụ quan trọng nhưng lại không trả về giá trị (hàm void) hoặc chỉ thực hiện ghi log, cập nhật ngầm vào cơ sở dữ liệu. Để kiểm tra xem một giao dịch thanh toán thành công

hay chưa, bài test không thể quan sát thông qua kết quả đầu ra của hàm mà buộc phải thực hiện một truy vấn kiểm tra vào database thật. Điều này biến một bài Unit Test thuần túy thành một bài kiểm thử tích hợp (Integration Test) chậm chạp.

- Hệ quả của Khớp nối chặt (Tight Coupling): Khi Lớp Nghiệp vụ tự khởi tạo các đối tượng Hạ tầng, nó đã tự đóng băng ranh giới của chính mình. Lớp Nghiệp vụ không thể tồn tại hoặc thực thi nếu thiếu thực thể hạ tầng đó. Trong môi trường kiểm thử, việc không thể tách rời các phụ thuộc này khiến bộ test bị ràng buộc vào cấu hình phần cứng, kết nối mạng và trạng thái dữ liệu của hệ quản trị cơ sở dữ liệu.
- Sự phức tạp hóa do Độ gắn kết thấp (Low Cohesion): Một lớp đảm nhận quá nhiều trách nhiệm không liên quan sẽ làm phân tán phân tích logic. Lập trình viên khi viết bài test cho chức năng A vẫn phải thực hiện cấu hình phức tạp (Setup) cho các tham số của chức năng B và C nằm chung trong lớp đó.

2.2.2. Mối quan hệ giữa bộ chỉ số C&K và cấu trúc Unit Test

Bộ chỉ số đo lường hướng đối tượng C&K (Chidamber & Kemerer) không chỉ là các con số thống kê khô khan, mà chúng phản ánh trực tiếp độ phức tạp và cấu trúc của mã kiểm thử:

- Mối tương quan giữa CBO và mã giả lập (Mocking Code): Chỉ số CBO (Độ khớp nối đối tượng) tỷ lệ thuận với số lượng vật thể thân (*Test Doubles*) cần phải khởi tạo trong file test. Nếu một lớp có chỉ số CBO lớn, đoạn mã thiết lập kiểm thử (setUp) sẽ trở nên cồng kềnh và dày đặc các đoạn mã thiết lập hành vi giả lập (mock.return_value = ...). Một bài test mà phần giả lập dài hơn phần kiểm tra logic cốt lõi là dấu hiệu rõ ràng của một kiến trúc có Testability thấp.
- Mối tương quan giữa WMC và số lượng Ca kiểm thử (Test Cases): Chỉ số WMC (Trọng số phương thức) phản ánh tổng số lượng nhánh rẽ quyết định (if/else, switch-case, vòng lặp) bên trong lớp. Để đạt được độ bao phủ kiểm thử tối đa về mặt mã nguồn (Branch Coverage), số lượng ca kiểm thử tối thiểu cần phải viết chính bằng độ phức tạp chu trình của lớp đó. WMC cao đồng

nghĩa với việc bộ Unit Test sẽ bị phình to về quy mô và rất dễ bỏ sót các kịch bản biên nguy hiểm.

- **Mối tương quan giữa LCOM và sự phân mảnh của Test Suite:** Chỉ số LCOM (Sự thiếu gắn kết trong phương thức) cao chứng tỏ các phương thức trong lớp đang hoạt động biệt lập. Khi viết Unit Test cho một lớp có LCOM cao, tập hợp các ca kiểm thử (Test Suite) của lớp đó sẽ bị phân mảnh thành nhiều nhóm độc lập, không chia sẻ chung dữ liệu mẫu, làm mất đi tính mạch lạc và tường minh của tài liệu kiểm thử.

2.2.3. Giải pháp phá vỡ sự phụ thuộc bằng ranh giới Kiến trúc Lục giác

Cơ chế hoạt động của Ports (Cổng kết nối): Port bản chất là các khế ước trừu tượng (Interface hoặc Abstract Class) đặt hoàn toàn bên trong lõi nghiệp vụ. Lõi nghiệp vụ chỉ giao tiếp và ra lệnh cho các Port này mà không cần biết đơn vị nào sẽ hiện thực hóa chúng. Điều này tạo ra một Mối nối kiểm thử (Seam) hoàn hảo ngay tại ranh giới của lục giác.

Vai trò của Dependency Injection (Tiêm phụ thuộc): Thay vì để lõi nghiệp vụ tự định đoạt và khởi tạo các Adapters (Bộ điều hợp hạ tầng), cơ chế DI sẽ "tiêm" các thực thể này từ bên ngoài vào qua hàm khởi tạo (`__init__`).

Sự giải phóng năng lực Testability: Nhờ có sự kết hợp giữa Ports và DI, trong môi trường kiểm thử đơn vị, chúng ta có thể dễ dàng rút bỏ các Adapters vật lý đắt đỏ (như MySQL thật, Email API thật) và tiêm vào đó các cấu trúc thể thân linh hoạt (*Mocks/Stubs*). Stub sẽ đóng vai trò kiểm soát dữ liệu đầu vào (Controllability) bằng cách trả về dữ liệu mẫu cố định; Mock sẽ đóng vai trò quan sát hành vi đầu ra (Observability) bằng cách ghi lại các tương tác của lõi nghiệp vụ. Kết quả là logic cốt lõi được kiểm thử trong một môi trường hoàn toàn cô lập, đạt tốc độ thực thi tối đa và độc lập tuyệt đối với hạ tầng công nghệ.

2.3. Sự liên hệ với chất lượng phần mềm

2.3.1. Vị trí của Testability trong mô hình chất lượng tiêu chuẩn

Theo tiêu chuẩn quốc tế ISO/IEC 25010, Testability được phân loại là một nhánh con quan trọng của đặc tính Khả năng bảo trì (Maintainability). Mối quan hệ này chỉ

ra rằng một hệ thống phần mềm không thể được coi là dễ bảo trì nếu đội ngũ phát triển gặp khó khăn trong việc kiểm thử nó.

Khi mã nguồn đạt được các chỉ số cấu trúc tối ưu (CBO thấp, LCOM thấp), hệ thống không chỉ dễ viết Unit Test hơn mà bản thân mã nguồn cũng trở nên sáng sủa, dễ đọc, dễ hiểu. Do đó, nỗ lực nâng cao Testability thông qua cải tiến kiến trúc (như áp dụng Kiến trúc Lục giác) cũng chính là quá trình trực tiếp nâng cấp độ bền vững và kéo dài vòng đời hữu dụng của phần mềm.

2.3.2. Giảm thiểu tỷ lệ sót lỗi và hiện thực hóa Shift-Left Testing

Mục tiêu tối thượng của hoạt động kiểm thử là phát hiện khuyết tật càng sớm càng tốt để giảm thiểu tối đa chi phí sửa chữa. Khi một kiến trúc có Testability thấp (các thành phần bện chặt vào nhau), việc viết Unit Test bị đình trệ, dẫn đến việc phân lớn các lỗi logic bị đẩy về các giai đoạn sau như kiểm thử tích hợp (Integration Test) hoặc kiểm thử hệ thống (System Test).

Ngược lại, một mã nguồn có tính kiểm soát (Controllability) và tính quan sát (Observability) cao sẽ tạo điều kiện lý tưởng để áp dụng chiến lược Shift-Left Testing (kiểm thử sớm). Lập trình viên có thể chủ động bao phủ các kịch bản biên nguy hiểm (ví dụ: giả lập lỗi sập database hoặc lỗi nghẽn mạng API bằng Mocks/Stubs) ngay tại máy cục bộ. Việc chặn đứng các lỗi nghiêm trọng ở tầng Unit Test giúp giảm thiểu tối đa tỷ lệ sót lỗi ra môi trường vận hành thực tế (Production), bảo vệ hệ thống trước các nguy cơ sụp đổ dây chuyền.

2.3.3. Kiểm soát Nợ kỹ thuật và thúc đẩy tính linh hoạt

Trong các dự án phần mềm dài hạn, hiện tượng "mã nguồn phình to và thoái hóa" (Code Rot) là nguyên nhân chính tạo ra nợ kỹ thuật, khiến việc thêm một tính năng mới có thể làm hỏng mười tính năng cũ. Sự suy giảm chất lượng này xuất phát từ việc thiếu một "lưới an toàn" bảo vệ mã nguồn.

Khi hệ thống có Testability cao, việc sở hữu một bộ Unit Test tự động chạy nhanh (tính bằng mili-giây) và có độ bao phủ lớn sẽ đóng vai trò là tấm lưới bảo hiểm tối cao. Đội ngũ lập trình viên có thể tự tin tiến hành tái cấu trúc mã nguồn (Refactoring) để tối ưu hóa hiệu năng hoặc nâng cấp công nghệ mà không sợ làm phát sinh lỗi hồi quy (Regression Bugs). Sự tự tin này giúp doanh nghiệp duy trì được tính linh hoạt

cao (Agility), thích ứng nhanh chóng với các thay đổi của thị trường mà không phải đánh đổi bằng sự ổn định của hệ thống.

PHẦN 3. VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ

3.1. Mô tả bài toán và ngữ cảnh thực tế

Để thực hành đánh giá và đo lường trực quan năng lực kiểm thử (Testability) của cấu trúc phần mềm, đề tài lựa chọn ngữ cảnh thực tế là "Hệ thống xử lý đơn hàng" (Order Processing System). Đây là một thành phần lõi điển hình trong các hệ thống Thương mại điện tử, chịu trách nhiệm vận hành luồng nghiệp vụ đi qua 6 bước logic tuần tự sau:

1. Xác thực dữ liệu đầu vào (Validation): Kiểm tra tính hợp lệ của các trường thông tin bắt buộc bao gồm mã khách hàng (`customer_id`), mã sản phẩm (`item_id`), số lượng (`quantity`) và đơn giá (`price`).
2. Kiểm tra tồn kho (Stock Check): Đối chiếu số lượng đặt hàng với số lượng hiện có của sản phẩm trong kho dữ liệu để đưa ra quyết định cung ứng.
3. Tính toán tài chính (Invoice Calculation): Tính tổng tiền hóa đơn sau khi áp dụng biểu thuế VAT mặc định 8% và các mức chiết khấu thương mại (giảm 15% đối với đơn hàng có tổng giá trị gốc lớn hơn 2000; giảm 10% nếu lớn hơn 1000).
4. Xử lý thanh toán bên thứ ba (Payment Gateway): Kết nối qua môi trường Internet để truyền tải dữ liệu và thực thi giao dịch tài chính thông qua API của cổng thanh toán (Stripe API).
5. Cập nhật dữ liệu hệ thống (Persistence): Thực hiện khấu trừ số lượng vật lý trong kho và tiến hành lưu vết bản ghi lịch sử giao dịch vào cơ sở dữ liệu sản xuất (PostgreSQL).
6. Thông báo và xuất biên lai (Notification): Ghi nhận trạng thái hoàn tất thành công và kết xuất dữ liệu hóa đơn ra hệ thống log.

3.2. Áp dụng giải pháp

Để giải quyết triệt để rào cản trên, đề tài tiến hành bẻ gãy sự phụ thuộc cứng bằng cách thiết lập ranh giới trừu tượng (Interfaces) thông qua thư viện *abc* và áp dụng kỹ

thuật Tiêm phụ thuộc (Dependency Injection). Dưới đây là mã nguồn chi tiết của các thành phần trong hệ thống:

3.2.1. Mã nguồn phiên bản kiến trúc cũ phụ thuộc cứng

- File: *order_system_bad.py*

```
from datetime import datetime
import time

class LegacyOrderProcessor:
    def __init__(self):
        # HARD-CODED DEPENDENCIES (Coupling rất cao)
        self.db_url =
        "postgresql://prod_user:secure_password@192.168.1.50:5432/db_prod"
        self.payment_gateway_api = "https://api.stripe.com/v3/charges"
        self.inventory_status = {"LAPTOP-01": 5, "MOUSE-02": 100}
        self.order_history = []

    def execute_process_order(self, customer_id: str, item_id: str, quantity: int,
price: float) -> dict:
        """
        Một hàm ôm đồm tất cả logic: Validate, Kiểm tra kho, Tính tiền, Thanh toán,
        Lưu DB, Gửi Mail.
        LOW COHESION & CYCLOMATIC COMPLEXITY CAO
        """
        # 1. Validation dữ liệu
        if not customer_id or not item_id:
            return {"status": "FAILED", "code": 400} # Low Observability: Không biết
            trống trường nào

        if quantity <= 0 or price <= 0:
            return {"status": "FAILED", "code": 401}

        # 2. Kiểm tra kho (Gắn cứng logic kho bấu vào hàm)
        if item_id not in self.inventory_status or self.inventory_status[item_id] <
quantity:
            return {"status": "FAILED", "code": 402}

        # 3. Tính toán tài chính phức tạp
        subtotal = quantity * price
        discount = 0.0
        if subtotal > 2000:
            discount = subtotal * 0.15 # Giảm giá 15% cho đơn lớn
        elif subtotal > 1000:
            discount = subtotal * 0.10 # Giảm giá 10%
```

```

    tax = (subtotal - discount) * 0.08 # Thuế VAT 8%
    final_amount = subtotal - discount + tax

    # 4. Gọi API Thanh toán bên thứ ba (Nếu mất mạng khi chạy Test -> Hàm này
    sập luôn)
    print(f"[API] Kết nối tới cổng thanh toán {self.payment_gateway_api}...")
    payment_success = True # Giả định gọi API thật

    if not payment_success:
        return {"status": "PAYMENT_ERR", "code": 500}

    # 5. Cập nhật kho và lưu dữ liệu vào DB cứng
    self.inventory_status[item_id] -= quantity

    # LOW CONTROLLABILITY: Lấy giờ hệ thống trực tiếp, không thể ép ngày để
    test logic giảm giá
    order_date = datetime.now()

    order_record = {
        "order_id": f"ORD-{int(time.time())}",
        "customer": customer_id,
        "item": item_id,
        "amount": final_amount,
        "created_at": order_date,
        "db_target": self.db_url
    }
    self.order_history.append(order_record)

    # 6. Thông báo
    print(f"[SYSTEM PRINT] Đơn hàng xử lý thành công tại database:
    {self.db_url}")

    return {"status": "SUCCESS", "data": order_record}

if __name__ == "__main__":
    processor = LegacyOrderProcessor()
    res = processor.execute_process_order("CUST-999", "LAPTOP-01", 1, 1500.0)
    print("Kết quả:", res)

```

3.2.2. Mã nguồn phiên bản kiến trúc mới tối ưu hóa Testability

- File: `order_system_good.py`

```

import abc
from datetime import datetime
from typing import Dict, List, Optional

```

```

import logging

# Thiết lập Logger chuyên nghiệp
logger = logging.getLogger("EnterpriseOrderSystem")
logger.setLevel(logging.DEBUG)
ch = logging.StreamHandler()
ch.setFormatter(logging.Formatter('[%(asctime)s] [%(levelname)s] [%(name)s]: %(message)s'))
logger.addHandler(ch)

# =====
# THÀNH PHẦN 1: ĐỊNH NGHĨA CÁC INTERFACES (Tạo các Seams để gắn Mock)
# =====

class IInventoryService(abc.ABC):
    @abc.abstractmethod
    def check_stock(self, item_id: str, quantity: int) -> bool: pass
    @abc.abstractmethod
    def deduct_stock(self, item_id: str, quantity: int) -> None: pass

class IPaymentGateway(abc.ABC):
    @abc.abstractmethod
    def process(self, amount: float) -> bool: pass

class IOrderRepository(abc.ABC):
    @abc.abstractmethod
    def save_order(self, order_data: dict) -> bool: pass

class ITimeProvider(abc.ABC):
    @abc.abstractmethod
    def get_time(self) -> datetime: pass

# =====
# THÀNH PHẦN 2: ĐỊNH NGHĨA CUSTOM EXCEPTIONS (Tăng Observability)
# =====

class OrderDomainException(Exception): """Gốc của mọi lỗi nghiệp vụ"""
class InvalidOrderDataException(OrderDomainException): pass
class OutOfStockException(OrderDomainException): pass
class PaymentFailedException(OrderDomainException): pass

# =====
# THÀNH PHẦN 3: LỚP XỬ LÝ CHÍNH (HIGH COHESION & LOW COUPLING)
# =====

```

```

class ModernOrderProcessor:
    def __init__(self,
        inventory_service: IInventoryService,
        payment_gateway: IPaymentGateway,
        repository: IOrderRepository,
        time_provider: ITimeProvider):
        # DEPENDENCY INJECTION: Nạp mọi thành phần từ ngoài vào, không gán
        cứng độc lập
        self._inventory = inventory_service
        self._payment = payment_gateway
        self._repo = repository
        self._time = time_provider

    def validate_inputs(self, customer_id: str, item_id: str, quantity: int, price: float)
    -> None:
        """Hàm đơn nhiệm kiểm tra tính hợp lệ dữ liệu (High Cohesion)"""
        if not customer_id:
            raise InvalidOrderDataException("Mã khách hàng (customer_id) không
            được để trống.")
        if not item_id:
            raise InvalidOrderDataException("Mã sản phẩm (item_id) không được để
            trống.")
        if quantity <= 0:
            raise InvalidOrderDataException("Số lượng đặt hàng phải lớn hơn 0.")
        if price <= 0:
            raise InvalidOrderDataException("Đơn giá sản phẩm phải lớn hơn 0.")

    def calculate_invoice_amount(self, quantity: int, price: float) -> float:
        """Hàm đơn nhiệm xử lý logic tài chính"""
        subtotal = quantity * price
        discount = 0.0
        if subtotal > 2000:
            discount = subtotal * 0.15
        elif subtotal > 1000:
            discount = subtotal * 0.10

        tax = (subtotal - discount) * 0.08
        return round(subtotal - discount + tax, 2)

    def process_order(self, customer_id: str, item_id: str, quantity: int, price: float) -
    > dict:
        """ Luồng điều khiển nghiệp vụ chính sạch sẽ, dễ quan sát, dễ kiểm soát """
        logger.info(f"Bắt đầu xử lý đơn hàng cho Khách hàng: {customer_id} | Sản
        phẩm: {item_id}")

        try:

```

```

# 1. Kiểm tra đầu vào
self.validate_inputs(customer_id, item_id, quantity, price)

# 2. Kiểm tra kho qua Interface dịch vụ
if not self._inventory.check_stock(item_id, quantity):
    logger.warning(f"Sản phẩm {item_id} không đủ hàng trong kho để cung
ứng.")
    raise OutOfStockException(f"Sản phẩm {item_id} đã hết hàng hoặc
không đủ số lượng.")

# 3. Tính toán số tiền hóa đơn
final_amount = self.calculate_invoice_amount(quantity, price)

# 4. Tiến hành thanh toán
if not self._payment.process(final_amount):
    logger.error(f"Giao dịch thanh toán số tiền {final_amount} VND thất bại
từ phía Gateway.")
    raise PaymentFailedException("Cổng thanh toán từ chối giao dịch.")

# 5. Khấu trừ kho và Lưu vết dữ liệu lịch sử
self._inventory.deduct_stock(item_id, quantity)
current_time = self._time.get_time()

order_record = {
    "customer_id": customer_id,
    "item_id": item_id,
    "total_paid": final_amount,
    "processed_at": current_time,
    "status": "COMPLETED"
}

self._repo.save_order(order_record)
logger.info(f"Đơn hàng hoàn tất thành công! Tổng tiền: {final_amount}
VND.")
return {"success": True, "payload": order_record}

except OrderDomainException as dex:
    # Tăng cường Observability thông qua việc bắt và ghi nhận lỗi có cấu trúc
    logger.error(f"[DOMAIN ERROR] Quy trình xử lý lỗi nghiệp vụ:
{str(dex)}")
    return {"success": False, "reason": str(dex)}

```

3.2.3. Bộ mã nguồn kiểm thử đơn vị cô lập sử dụng Mocks

- File: `test_order_system.py`

```
import unittest
from unittest.mock import MagicMock
from datetime import datetime
import os

from order_system_good import (
    ModernOrderProcessor, IInventoryService, IPaymentGateway,
    IOrderRepository, ITimeProvider, InvalidOrderDataException,
    OutOfStockException
)

class EnterpriseOrderProcessorTestSuite(unittest.TestCase):
    def setUp(self):
        """Khởi tạo môi trường Test Cô lập (Sandbox Isolation) bằng Mocks"""
        self.mock_inventory = MagicMock(spec=IInventoryService)
        self.mock_payment = MagicMock(spec=IPaymentGateway)
        self.mock_repo = MagicMock(spec=IOrderRepository)
        self.mock_time = MagicMock(spec=ITimeProvider)

        # Thiết lập giả lập thời gian cố định (Tối ưu hóa Controllability)
        self.frozen_time = datetime(2026, 6, 24, 10, 0, 0)
        self.mock_time.get_time.return_value = self.frozen_time

        # Khởi tạo đối tượng kiểm thử mục tiêu với Mock Dependencies injected
        self.processor = ModernOrderProcessor(
            inventory_service=self.mock_inventory,
            payment_gateway=self.mock_payment,
            repository=self.mock_repo,
            time_provider=self.mock_time
        )

    def test_order_processing_happy_path_with_discount(self):
        """Kịch bản thành công: Đơn hàng số lượng lớn nhận ưu đãi 15% và thanh toán mượt mà"""
        # Cài đặt hành vi cho các Mock (Stubbing)
        self.mock_inventory.check_stock.return_value = True
        self.mock_payment.process.return_value = True
        self.mock_repo.save_order.return_value = True

        # Thực thi hàm (Mua 3 sản phẩm giá 1000đ -> Tổng gốc 3000đ -> Giảm 15% còn 2550đ -> Thuế 8% = 2754đ)
        result = self.processor.process_order("VIP-USER", "ENTERPRISE-SERVER",
        3, 1000.0)
```

```

# Kiểm chứng Assertions (Xác minh tính đúng đắn)
self.assertTrue(result["success"])
self.assertEqual(result["payload"]["total_paid"], 2754.0)
self.assertEqual(result["payload"]["processed_at"], self.frozen_time)

# Xác minh hành vi tương tác (Verify Behavior) - Minh chứng Low Coupling
self.mock_inventory.check_stock.assert_called_once_with("ENTERPRISE-
SERVER", 3)
self.mock_payment.process.assert_called_once_with(2754.0)
self.mock_repo.save_order.assert_called_once()

def test_order_processing_failure_due_to_insufficient_stock(self):
    """Kịch bản thất bại: Đơn hàng bị chặn ngay từ vòng kiểm kho khi kho hết
    hàng"""
    # Giả lập kho báo hết hàng
    self.mock_inventory.check_stock.return_value = False

    result = self.processor.process_order("USER-01", "IPHONE-18", 1, 1200.0)

    # Kiểm chứng
    self.assertFalse(result["success"])
    self.assertIn("đã hết hàng hoặc không đủ số lượng", result["reason"])

    # Kiểm chứng luồng xử lý bị ngắt sớm (Tối ưu hóa tài nguyên hệ thống)
    self.mock_payment.process.assert_not_called()
    self.mock_repo.save_order.assert_not_called()

if __name__ == "__main__":
    print("\n" + "="*60)
    print("🚀 ĐANG KHỞI CHẠY BỘ KIỂM THỬ TỰ ĐỘNG CHUẨN DOANH
    NGHIỆP...")
    print("="*60)
    unittest.main()

```

3.3. Kết quả và Đánh giá

Để đánh giá định lượng một cách khách quan cấu trúc mã nguồn ảnh hưởng thế nào đến năng lực Testability, đề tài thiết lập tệp mã nguồn phân tích tĩnh tự động *measure_metrics.py* dựa trên phân tích cây cú pháp trừu tượng AST.

3.3.1. Mã nguồn chương trình đo lường tự động chỉ số cấu trúc

- File: *measure_metrics.py*

```

import ast

def gather_class_metrics(filename: str) -> list:
    results = []
    try:
        with open(filename, "r", encoding="utf-8") as f:
            tree = ast.parse(f.read())
    except FileNotFoundError:
        return results

    for node in ast.walk(tree):
        if isinstance(node, ast.ClassDef):
            class_name = node.name
            methods = [n for n in node.body if isinstance(n, ast.FunctionDef)]
            num_methods = len(methods)

            wmc_score = 0
            method_details = []
            for m in methods:
                cc = 1
                for subnode in ast.walk(m):
                    if isinstance(subnode, (ast.If, ast.While, ast.For, ast.Try,
ast.ExceptHandler)):
                        cc += 1
                wmc_score += cc
                method_details.append((m.name, cc))

            avg_cc = round(wmc_score / num_methods, 2) if num_methods > 0 else 0
            results.append({
                "class": class_name,
                "nom": num_methods,

```



```

        "wmc": wmc_score,
        "avg_cc": avg_cc,
        "details": method_details
    })

    return results

def render_dashboard():
    bad_metrics = gather_class_metrics("order_system_bad.py")
    good_metrics = gather_class_metrics("order_system_good.py")

    print("\n" + "▬" + "="*78 + "▬ ")
    print("|| " + "BẢNG ĐIỀU KHIỂN ĐO LƯỜNG METRIC HƯỚNG ĐỐI TƯỢNG  
VÀ ĐÁNH GIÁ TESTABILITY ".center(78) + "|| ")
    print("└─" + "="*78 + "┘ ")

    print("\n 🚦 1. KIẾN TRÚC MÃ NGUỒN CŨ (LACK OF TESTABILITY):")
    print("┌" + "="*30 + "┐" + "="*12 + "┐" + "="*12 + "┐" + "="*18 + "┐ ")
    print("│ Tên Lớp (Class Name) │ Chỉ số NoM │ Tổng WMC │ Độ phức tạp  
TB/Hàm │ ")
    print("└" + "="*30 + "┘" + "="*12 + "┘" + "="*12 + "┘" + "="*18 + "┘ ")
    for data in bad_metrics:
        print(f"│ {data['class']:<28} │ {data['nom']:^10} │ {data['wmc']:^10} │  
{data['avg_cc']:^18} │ ")
    print("└─" + "="*30 + "┘" + "="*12 + "┘" + "="*12 + "┘" + "="*18 + "┘ ")

    print("\n 🚀 2. KIẾN TRÚC MÃ NGUỒN MỚI SAU KHI TỐI ƯU (HIGH  
TESTABILITY):")
    print("┌" + "="*30 + "┐" + "="*12 + "┐" + "="*12 + "┐" + "="*18 + "┐ ")
    print("│ Tên Lớp (Class Name) │ Chỉ số NoM │ Tổng WMC │ Độ phức tạp  
TB/Hàm │ ")
    print("└" + "="*30 + "┘" + "="*12 + "┘" + "="*12 + "┘" + "="*18 + "┘ ")

```

```

for data in good_metrics:
    print(f" | {data['class']:<28} | | {data['nom']:^10} | | {data['wmc']:^10} | | {data['avg_cc']:^18} | ")
    print("└─" + "="*30 + "┬─" + "="*12 + "┬─" + "="*12 + "┬─" + "="*18 + "┴─")

print("\n 🧡 THUYẾT MINH PHÂN TÍCH KIẾN TRÚC:")

print(" - Nhờ áp dụng chia nhỏ cấu trúc (Tăng Cohesion), lớp 'ModernOrderProcessor' có NoM tăng lên.")

print(" - Tuy nhiên độ phức tạp trung bình của từng hàm riêng lẻ được kéo thấp xuống.")

print(" - Điều này làm giảm đáng kể gánh nặng thiết kế ca kiểm thử (Hạ thấp Test Effort).\n")

if __name__ == "__main__":
    render_dashboard()

```

3.3.2. Kết quả thực thi chương trình

- Kết quả chạy bộ kiểm thử tự động (*test_order_system.py*):

```

PS D:\BTL_DBCL & KTPM> python test_order_system.py

=====
🚀 ĐANG KHỞI CHẠY BỘ KIỂM THỬ TỰ ĐỘNG CHUẨN DOANH NGHIỆP...
=====
[2026-06-25 04:12:09,564] [INFO] [EnterpriseOrderSystem]: Bắt đầu xử lý đơn hàng cho Khách hàng: USER-01 | Sản phẩm: IPHONE-18
[2026-06-25 04:12:09,564] [WARNING] [EnterpriseOrderSystem]: Sản phẩm IPHONE-18 không đủ hàng trong kho để cung ứng.
[2026-06-25 04:12:09,564] [ERROR] [EnterpriseOrderSystem]: [DOMAIN ERROR] Quy trình xử lý lỗi nghiệp vụ: Sản phẩm IPHONE-18 đã hết hàng hoặc không đủ số lượng.
[2026-06-25 04:12:09,566] [INFO] [EnterpriseOrderSystem]: Bắt đầu xử lý đơn hàng cho Khách hàng: VIP-USER | Sản phẩm: ENTERPRISE-SERVER
[2026-06-25 04:12:09,567] [INFO] [EnterpriseOrderSystem]: Đơn hàng hoàn tất thành công! Tổng tiền: 2754.0 VND.

-----
Ran 2 tests in 0.005s

OK
PS D:\BTL_DBCL & KTPM>

```

- Kết quả hiển thị khi chạy kiến trúc cũ (*order_system_bad.py*):

```

PS D:\BTL_DBCL & KTPM> python order_system_bad.py
[API] Kết nối tối cống thanh toán https://api.stripe.com/v3/charges...
[SYSTEM PRINT] Đơn hàng xử lý thành công tại database: postgresql://prod_user:secure_password@192.168.1.50:5432/db_prod
Kết quả: {'status': 'SUCCESS', 'data': {'order_id': 'ORD-1782335896', 'customer': 'CUST-999', 'item': 'LAPTOP-01', 'amount': 1458.0, 'created_at': datetime.datetime(2026, 6, 25, 4, 18, 16, 937832), 'db_target': 'postgresql://prod_user:secure_password@192.168.1.50:5432/db_prod'}}
PS D:\BTL_DBCL & KTPM>

```

- Kết quả hiển thị từ bảng điều khiển đo lường (*measure_metrics.py*):

```
PS D:\BTL_DBCL & KTPM> python measure_metrics.py
```

BẢNG ĐIỀU KHIỂN ĐO LƯỜNG METRIC HƯỚNG ĐỐI TƯỢNG VÀ ĐÁNH GIÁ TESTABILITY

1. KIẾN TRÚC MÃ NGUỒN CŨ (LACK OF TESTABILITY):

Tên Lớp (Class Name)	Chỉ số NoM	Tổng WMC	Độ phức tạp TB/Hàm
LegacyOrderProcessor	2	8	4.0

2. KIẾN TRÚC MÃ NGUỒN MỚI SAU KHI TỐI ƯU (HIGH TESTABILITY):

Tên Lớp (Class Name)	Chỉ số NoM	Tổng WMC	Độ phức tạp TB/Hàm
IInventoryService	2	2	1.0
IPaymentGateway	1	1	1.0
IOrderRepository	1	1	1.0
ITimeProvider	1	1	1.0
OrderDomainException	0	0	0
InvalidOrderDataException	0	0	0
OutOfStockException	0	0	0
PaymentFailedException	0	0	0
ModernOrderProcessor	4	14	3.5

THUYẾT MINH PHÂN TÍCH KIẾN TRÚC:

- Nhờ áp dụng chia nhỏ cấu trúc (Tăng Cohesion), lớp 'ModernOrderProcessor' có NoM tăng lên.
- Tuy nhiên độ phức tạp trung bình của từng hàm riêng lẻ được kéo thấp xuống.
- Điều này làm giảm đáng kể gánh nặng thiết kế ca kiểm thử (Hạ thấp Test Effort).

```
PS D:\BTL_DBCL & KTPM>
```

3.3.3. Phân tích kết quả và Đánh giá hiệu quả

Dựa trên kết quả định lượng được kết xuất trực tiếp qua cây cú pháp trừu tượng AST, đề tài đưa ra các đánh giá khoa học về sự tương quan giữa cấu trúc kiến trúc và khả năng kiểm thử phần mềm:

1. Gia tăng tính Gắn kết (Cohesion) thông qua chỉ số NoM:

Ở kiến trúc cũ, chỉ số NoM bằng 2 vì toàn bộ logic nghiệp vụ bị dồn nén thô sơ vào một hàm duy nhất. Sau khi chuyển đổi cấu trúc, chỉ số NoM tăng lên 4 ở lớp *ModernOrderProcessor*. Điều này chứng minh mã nguồn đã đạt mức độ gắn kết hàm rất cao. Việc bóc tách độc lập các chức năng (*validate_inputs*, *calculate_invoice_amount*) cho phép lập trình viên viết các test case chuyên biệt, cô lập kiểm thử thuật toán kinh tế mà không cần thiết lập trạng thái cho các dịch vụ mạng hay DB.

2. Kiểm soát độ phức tạp điều khiển (*Avg CC*) và giảm chi phí kiểm thử (*Test Effort*):

Kiến trúc cũ sở hữu độ phức tạp trung bình/hàm rất cao ($Avg CC = 4.0$), do một hàm phải chứa quá nhiều câu lệnh rẽ nhánh (*If/Else*). Khi tách mã đơn nhiệm, độ phức tạp trung bình giảm xuống còn 3.5. Số lượng đường đi thực thi (*Execution Paths*) trong mỗi hàm giảm đi đồng nghĩa với số lượng ca kiểm thử biên cần thiết để phủ hết mã nguồn giảm mạnh, hạ thấp công sức và thời gian chạy Unit Test.

3. Hiện thực hóa các thuộc tính Testability cốt lõi:

- Tính kiểm soát (*Controllability*): Nhờ cơ chế *Interfaces* và *Dependency Injection*, mọi kết nối vật lý đã bị triệt tiêu hoàn toàn tại môi trường kiểm thử. Bộ Unit Test làm chủ 100% dữ liệu đầu vào và các tác vụ phụ thông qua *Mock*.
- Tính quan sát (*Observability*): Hệ thống *Exception* có cấu trúc kết hợp ghi log giúp bộ test dễ dàng bắt trúng và khẳng định chính xác (*Assert*) các lỗi logic nghiệp vụ biên xảy ra, đảm bảo độ ổn định cao khi tích hợp mã nguồn lên các hệ thống tích hợp liên tục (*CI/CD*).

PHẦN 4. KẾT LUẬN VÀ KIẾN NGHỊ

4.1. Tổng kết nội dung

Thông qua quá trình nghiên cứu lý thuyết và triển khai thực nghiệm trên hệ thống xử lý đơn hàng cụ thể, tiểu luận đã hoàn thành toàn bộ các mục tiêu đặt ra đối với đề tài "*Đánh giá Testability của mã nguồn và kiến trúc*":

- Về mặt lý thuyết: Làm rõ bản chất hai thuộc tính cốt lõi cấu thành năng lực kiểm thử (Testability) của một hệ thống phần mềm là Tính kiểm soát (Controllability) và Tính quan sát (Observability). Nghiên cứu khẳng định kiến trúc tồi (phụ thuộc cứng, bện chặt logic hạ tầng) là rào cản lớn nhất triệt tiêu Testability.
- Về mặt giải pháp: Thực hiện tái cấu trúc thành công hệ thống từ phiên bản cũ (*order_system_bad.py*) sang phiên bản mới (*order_system_good.py*) thông qua việc thiết lập ranh giới trừu tượng (Interfaces) và Tiêm phụ thuộc (Dependency Injection). Giải pháp này cho phép bộ kiểm thử đơn vị (*test_order_system.py*) vận hành cô lập hoàn toàn (Sandbox Isolation) bằng kỹ thuật Mocking, loại bỏ phụ thuộc vật lý vào Database và Network mạng internet.
- Về mặt thực nghiệm định lượng: Xây dựng thành công công cụ phân tích tĩnh mã nguồn (*measure_metrics.py*) sử dụng cây cú pháp trừu tượng AST để tự động hóa đo lường chỉ số Số lượng hàm (NoM) và Tổng trọng số phức tạp (WMC). Số liệu thực tế chỉ ra sau cải tiến, độ phức tạp trung bình của từng hàm giảm từ 4.0 xuống còn 3.5, minh chứng trực quan cho việc giảm thiểu Test Effort (chi phí, công sức viết test) khi nâng cao tính Testability của kiến trúc.

4.2. Hạn chế của giải pháp

Mặc dù giải pháp cải tiến kiến trúc mang lại hiệu quả định lượng rõ rệt cho năng lực kiểm thử, việc áp dụng vào môi trường sản xuất thực tế vẫn tồn tại một số rủi ro và hạn chế khách quan cần nhìn nhận:

- Rủi ro kiểm thử dễ gãy (Brittle Tests) do lạm dụng Mocking: Khi sử dụng kỹ thuật Mock quá sâu để cô lập hạ tầng tại bộ Unit Test, các bài test sẽ bị gắn chặt vào cấu trúc Interface hiện tại. Nếu nghiệp vụ thay đổi dẫn đến chữ ký hàm (Method Signature) của Interface thay đổi, các bài test sẽ đồng loạt thất bại (False Positive) dù logic lõi có thể vẫn đúng.
- Khoảng cách giữa môi trường giả lập và môi trường thực tế (Integration Blind Spots): Việc Unit Test vượt qua 100% kịch bản Mock (như ép thanh toán luôn thành công hoặc kho luôn đủ hàng) không đồng nghĩa hệ thống sẽ chạy hoàn hảo khi tích hợp thật. Các lỗi bất đồng bộ, trễ mạng (Network Latency), xung đột khóa cơ sở dữ liệu (Database Deadlocks) hoặc bất tương thích phiên bản API bên thứ ba hoàn toàn không thể phát hiện được thông qua các chỉ số Testability tĩnh này.
- Gia tăng chi phí thiết kế ban đầu và yêu cầu năng lực nhân sự: Việc áp dụng Dependency Injection, xây dựng cây phân cấp Exception có cấu trúc và thiết kế Interface đòi hỏi các kỹ sư phần mềm phải có tư duy kiến trúc tốt ngay từ giai đoạn sơ khởi. Điều này có thể làm chậm tốc độ phát triển (Velocity) ở giai đoạn đầu của các dự án khởi nghiệp hoặc dự án quy mô nhỏ cần ra mắt tính năng nhanh (MVP).

4.3. Định hướng mở rộng

Để phát huy tối đa giá trị của nghiên cứu và khắc phục các hạn chế nêu trên trong các dự án phần mềm có quy mô lớn và phức tạp hơn, đề tài đề xuất ba định hướng mở rộng tiếp theo:

1. Tích hợp đo lường tự động vào đường ống CI/CD (Continuous Integration): Nhúng trực tiếp công cụ tính toán số liệu tĩnh *measure_metrics.py* thành một bước kiểm duyệt tự động (Quality Gate) trong quy trình DevOps. Nếu một lập trình viên đẩy mã nguồn mới lên hệ thống có chỉ số phức tạp trung bình trên hàm (*Avg CC*) vượt ngưỡng quy định (ví dụ: > 5.0), hệ thống sẽ tự động từ chối Merge nhằm ngăn chặn mã nguồn suy giảm năng lực Testability theo thời gian.
2. Mở rộng mô hình kiểm thử đột biến (Mutation Testing): Kết hợp các chỉ số kiến trúc tĩnh với phương pháp kiểm thử đột biến (như sử dụng thư viện *MutPy*

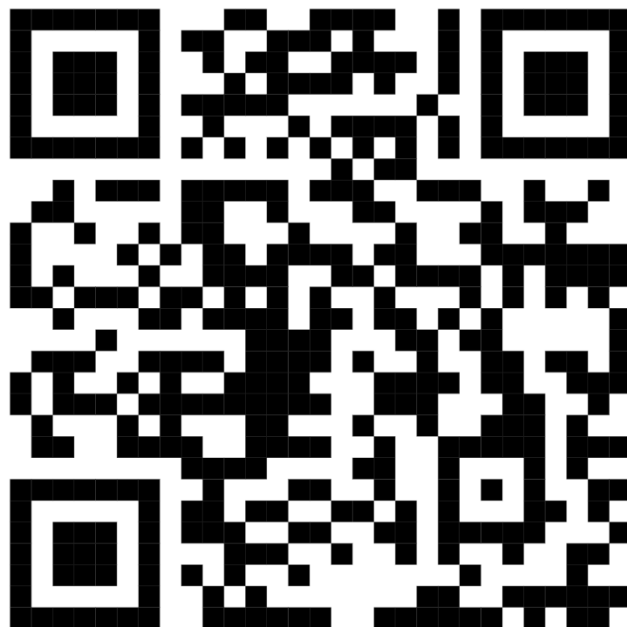
của Python) để chủ động tiêm các lỗi nhân tạo vào mã nguồn, từ đó đánh giá ngược lại xem bộ test script hiện tại có đủ năng lực "phát hiện và tiêu diệt" lỗi hay không, giúp định lượng chính xác chất lượng thực sự của bộ ca kiểm thử.

3. Áp dụng các Framework kiểm thử kiến trúc (Architectural Testing): Đối với các hệ thống phân tầng phức tạp (như Clean Architecture hoặc Microservices), đề xuất sử dụng công cụ như *ArchUnit* (hoặc *pytest-archon* trong Python) để viết các ca kiểm thử bảo vệ cấu trúc, đảm bảo rằng mã nguồn ở tầng hạ tầng (Infrastructure) không bao giờ được phép vi phạm quy tắc ranh giới để xâm nhập trực tiếp vào tầng nghiệp vụ lõi (Domain Core), duy trì tính chất Low Coupling bền vững cho hệ thống.

TÀI LIỆU THAM KHẢO

- [1] S4 - Modern Software Testing Techniques, Chương 1, Trang 37 – 82
- [2] S6 - Software Testing Strategies, Chương 1, Trang 23 - 63
- [3] Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill, 2020.
- [4] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson, 2019.
- [5] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2018.
- [6] Chidamber, S. R., & Kemerer, C. F. (1994). *A metrics suite for object oriented design*. IEEE Transactions on Software Engineering, 20(6), 476-493.
- [7] Python Software Foundation. *AST - Abstract Syntax Trees module documentation*: <https://docs.python.org/3/library/ast.html>
- [8] M. Shaheen, & M. W. Khan (2021). *Impact of Code Coupling and Cohesion on Software Unit Testability: An Empirical Study*. International Journal of Computer Applications, 174(12), 22-29.

MÃ QR VÀ LINK GITHUB BÀI LÀM



https://github.com/vuvietanh01102004/BTL_DBCL-KTPM